

AIA UNIT 1 Notes

1. Introduction to Artificial Intelligence

1.1 What is Artificial Intelligence?

Artificial Intelligence (AI) is the branch of computer science that focuses on building machines capable of performing tasks that normally require human intelligence. These tasks include learning, reasoning, problem solving, perception, and understanding language.

Three core cognitive processes AI focuses on:

- **Learning Processes:** Gathering data and creating rules (algorithms) that transform raw data into useful information. The algorithms provide step-by-step instructions for accomplishing a task.
- **Reasoning Processes:** Selecting the best algorithm to achieve a desired result — choosing the optimal path toward a goal.
- **Self-Correction Processes:** Continuously fine-tuning algorithms to ensure they produce the most reliable and accurate results over time.

Key Definition

AI = machines that simulate human cognitive abilities: learning, reasoning, problem-solving, perception, and language understanding.

1.2 History of Artificial Intelligence

Understanding the history helps you answer questions about milestones and why each era matters.

Era	Key Milestone	Significance
1940s–1950s	Alan Turing's 1950 paper; Turing Test proposed	Laid theoretical foundation — can machines think?
1956	Dartmouth Conference — term 'AI' coined	Formal birth of AI as a discipline
1960s–1970s	Rule-based systems; ELIZA chatbot built	Early NLP; showed machines could mimic conversation
1980s	Expert Systems (medicine, finance, engineering)	Knowledge-based decision-making enters industry
1990s–2000s	Machine Learning emerges; IBM Deep Blue beats Kasparov (1997)	Data-driven AI; strategic game-playing milestone
2010s–Present	Deep Learning revolution; Siri, GPT, autonomous cars	AI embedded in daily life across every industry

1.3 Classification of AI

1.3.1 Narrow AI (Weak AI)

Designed and trained for one specific task. All current AI systems fall in this category.

- Examples: Siri, Google Translate, recommendation engines, chess engines.

1.3.2 General AI (Strong AI)

A hypothetical system that can perform any intellectual task as efficiently as a human — it can think and reason across any domain by itself.

- No system with true General AI exists today.
- Researchers worldwide are actively working toward it.

1.3.3 Super AI

A level of intelligence where machines surpass human intelligence in every aspect — including creativity, social intelligence, and problem-solving.

- Still entirely hypothetical; considered a transformative (and potentially risky) milestone.
- Key characteristics: autonomous thinking, reasoning, puzzle-solving, judgment, planning, learning, and communication.
- It is considered an outcome of successfully achieving General AI.

AI Hierarchy

Narrow AI (today) → General AI (theoretical) → Super AI (hypothetical future). Similarly: AI ⊃ Machine Learning ⊃ Deep Learning.

1.4 Current Trends & Applications of AI

- **Healthcare:** Medical imaging analysis, drug discovery, predictive diagnostics, cancer and heart-condition detection.
- **Finance:** Fraud detection, algorithmic trading, credit scoring, automated customer service.
- **Transportation:** Self-driving cars (Tesla, Waymo), traffic prediction, fleet optimisation.
- **Manufacturing:** Predictive maintenance, quality control, robotic automation.
- **Education:** Personalised learning, automated grading, intelligent tutoring systems.
- **NLP / Chatbots:** Language translation, sentiment analysis, content creation, customer support.
- **Agriculture:** Crop monitoring, yield prediction, automated irrigation.
- **Cybersecurity:** Threat detection, anomaly analysis, automated incident response.

2. Intelligent Agents

2.1 What is an Agent?

An agent is anything that perceives its environment through sensors and acts upon that environment through actuators.

- Human Agent: eyes, ears (sensors) → hands, vocal cords (actuators)
- Robotic Agent: cameras, IR sensors → motors, grippers
- Software Agent: keystrokes, file inputs → screen outputs, file writes

2.2 Properties of a Rational Agent

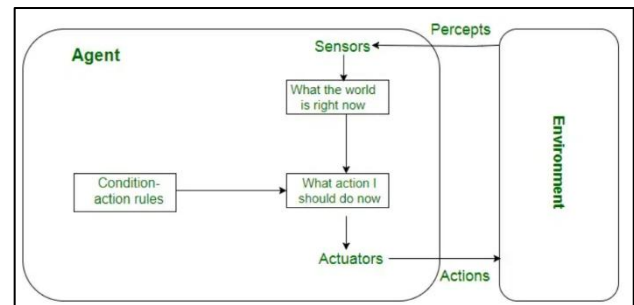
- **Rationality:** Chooses actions that maximise the expected performance measure given the percept sequence and built-in knowledge.
- **Autonomy:** Relies on own percepts rather than built-in assumptions about the environment.
- **Proactiveness:** Takes initiative to meet goals — doesn't just react.
- **Social Ability:** Can interact with other agents or humans.

2.3 Types of AI Agents

Type 1 — Simple Reflex Agent

Acts only on the current percept, ignoring the full history. Uses condition–action rules (if-then rules).

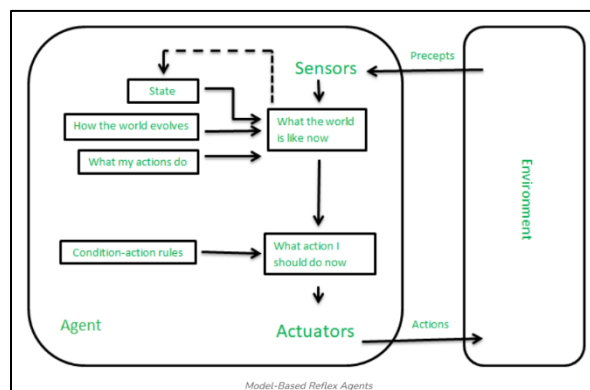
- **Strength:** Simple and fast.
- **Weakness:** Fails in partially observable environments; has no memory.
- Example: A thermostat that turns on heating when temperature < threshold.



Type 2 — Model-Based Reflex Agent

Maintains an internal model (state) of the world to handle partial observability. Updates state based on how the world evolves and how its own actions affect it.

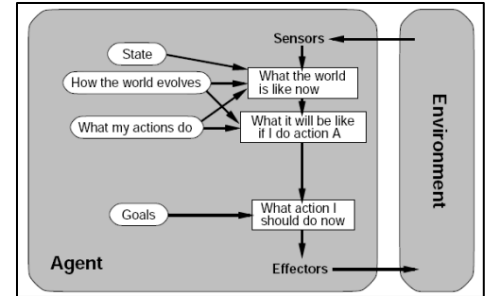
- Example: A car with sensors that tracks whether the lane ahead is clear even when a vehicle temporarily disappears from sensor range.



Type 3 — Goal-Based Agent

Has a goal and chooses actions that lead to achieving it. Combines the world model with goal information to decide what to do.

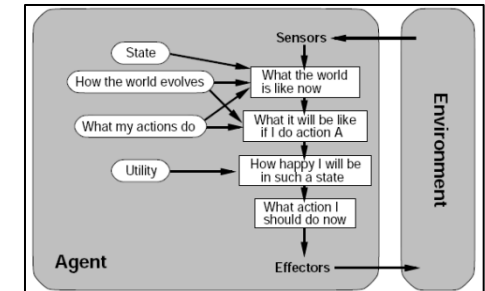
- More flexible than reflex agents — can change behaviour by changing the goal.
- Example: A GPS navigation system choosing a route to reach a destination.



Type 4 — Utility-Based Agent

Uses a utility function to measure how 'happy' a state is. Chooses actions that maximise expected utility, allowing trade-offs between conflicting goals.

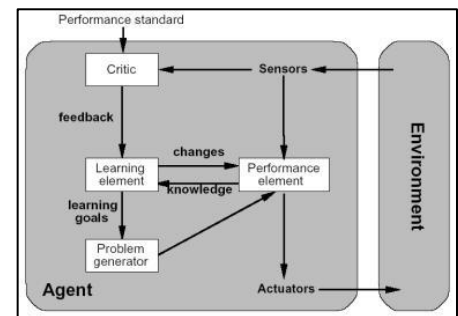
- Example: Self-driving car balancing speed, safety, comfort, and fuel efficiency.



Type 5 — Learning Agent

Improves its performance over time by learning from the environment. Consists of four components:

- **Learning Element:** Responsible for making improvements (modifying knowledge/rules).
- **Performance Element:** Selects actions based on current knowledge.
- **Critic:** Evaluates how well the agent is performing.
- **Problem Generator:** Suggests new exploratory actions.
- Example: Recommendation engines, game-playing agents like AlphaGo.



2.4 Environment Types (Important for Exam!)

Property	Type A	Type B
Observability	Fully Observable (agent sees entire state)	Partially Observable (limited sensors)
Agents	Single-agent	Multi-agent (cooperative or competitive)
Outcome	Deterministic (actions have predictable results)	Stochastic (uncertain outcomes)
Time	Static (environment doesn't change while deciding)	Dynamic (changes while agent is thinking)
State Space	Discrete (finite states)	Continuous (infinite states, e.g. robot position)
Episodes	Episodic (each action independent)	Sequential (current action affects future)

3. Problem Solving by Search

3.1 Defining a Search Problem

A search problem has five components:

- **Initial State:** The state from which the agent starts.
- **Actions:** The set of possible moves/operators available.
- **Transition Model:** Describes what each action does — $\text{RESULT}(s, a) = s'$.
- **Goal Test:** Determines whether a given state is a goal state.
- **Path Cost:** A numeric cost assigned to each path; the agent seeks the lowest-cost path.

3.2 Uninformed (Blind) Search

These strategies have no additional information about states beyond the problem definition — they explore systematically without any hints about proximity to the goal.

3.2.1 Depth First Search (DFS)

Explores as far as possible along each branch before backtracking. Uses a LIFO stack (or recursion).

Property	Details
Data Structure	Stack (LIFO)
Completeness	Not complete for infinite/cyclic graphs (without visited check)
Optimality	NOT optimal — finds a solution, not necessarily the shortest path
Time Complexity	$O(b^m)$ — b = branching factor, m = max depth
Space Complexity	$O(b \times m)$ — only stores the current path

- **How it works:** Push initial state. Pop top node. If goal, stop. Else expand and push children. Repeat.
- **Best for:** Deep solutions, memory-constrained problems.
- **Problem:** Can get stuck in infinite loops if cycles exist.

► **DFS Video Reference:** <https://www.youtube.com/watch?v=dWpvK-8uAqM>

3.2.2 Breadth First Search (BFS)

Explores all nodes at the current depth before moving deeper. Uses a FIFO queue.

Property	Details
Data Structure	Queue (FIFO)
Completeness	Complete — always finds a solution if one exists
Optimality	Optimal when all step costs are equal (finds shallowest/shortest path)
Time Complexity	$O(b^d)$ — d = depth of the shallowest goal

Property	Details
Space Complexity	$O(b^d)$ — stores all nodes at the current depth level

- **How it works:** Enqueue initial state. Dequeue front node. If goal, stop. Else expand and enqueue all children.
- **Best for:** Shortest-path problems in unweighted graphs.
- **Problem:** High memory usage — stores the entire frontier.

► **BFS Video Reference:** <https://www.youtube.com/watch?v=SZ2a668G6vA>

DFS vs BFS — Quick Comparison

DFS: deep, low memory, not optimal. BFS: wide, high memory, optimal (equal costs).
Remember: D=Deep/Stack, B=Broad/Queue.

3.3 Informed (Heuristic) Search

Uses problem-specific knowledge (a heuristic function $h(n)$) to estimate the cost from the current node to the goal — guiding search toward the goal more efficiently.

Heuristic function $h(n)$: estimated cost from node n to the goal.

3.3.1 Greedy Best First Search

Expands the node that appears closest to the goal according to $h(n)$ alone.

- **Evaluation function:** $f(n) = h(n)$
- **Completeness:** Not complete (can follow an infinitely wrong path).
- **Optimality:** Not optimal.
- **Speed:** Fast in practice — often reaches goal quickly.

Think of it as: always go toward what looks closest — like following a compass without considering what's in the way.

3.3.2 A* Search (A-Star)

Combines actual cost $g(n)$ (cost to reach n from start) with heuristic $h(n)$ (estimated cost to goal).

$$f(n) = g(n) + h(n)$$

- A* is both complete and optimal if $h(n)$ is admissible (never over-estimates the true cost).
- An admissible heuristic: $h(n) \leq$ actual cost to reach goal.
- Example: In map routing, $h(n)$ = straight-line distance is admissible because you can't travel faster than a straight line.

Algorithm	Function	Optimal?
BFS (uniform cost)	$f(n) = g(n)$	Yes (equal costs)
Greedy Best First	$f(n) = h(n)$	No
A*	$f(n) = g(n) + h(n)$	Yes (admissible h)

3.4 Local Search

3.4.1 Hill Climbing

A local search algorithm that continuously moves in the direction of increasing value (or decreasing cost) — like climbing a hill to reach the peak.

Algorithm:

1. Start with an initial state.
 2. Evaluate all neighbours.
 3. Move to the best neighbour (higher value / lower cost).
 4. Repeat until no neighbour is better → local optimum reached.
- **Problem 1 — Local Maxima:** Gets stuck at a peak that is not the global maximum.
 - **Problem 2 — Plateau:** Flat area where no neighbour is better.
 - **Problem 3 — Ridge:** Sequence of local maxima that is hard to navigate.

Variants to overcome these problems:

- **Stochastic Hill Climbing:** Randomly selects from uphill moves.
- **Random-Restart Hill Climbing:** Restarts from random initial states multiple times.
- **Simulated Annealing:** Occasionally allows downhill moves (with decreasing probability over time) to escape local maxima.

Hill Climbing Analogy

Imagine climbing a foggy mountain — you can only see your immediate surroundings. You always step uphill, but you might end up on a small hill instead of the tallest peak.

3.5 Problem Reduction Search

3.5.1 AND-OR Graphs and AO* Algorithm

Standard search assumes a problem is solved by a single path. Problem reduction decomposes a problem into sub-problems using AND-OR trees:

- **OR node:** The problem can be solved by solving ANY ONE of the sub-problems (alternatives).
- **AND node:** The problem requires ALL sub-problems to be solved (decomposition).

AO* (AND-OR A*) Algorithm:

1. Start at the root (original problem).
 2. Expand a node — determine if it is an OR node (pick best) or AND node (solve all).
 3. Apply a heuristic to estimate cost of each sub-problem.
 4. Propagate costs upward — AND node cost = sum of children; OR node cost = min of children.
 5. Mark solved nodes and backtrack to find the optimal solution graph.
- **Use case:** Theorem proving, game trees, planning problems where sub-goals must all be achieved.

- **Advantage over A*:** Handles non-deterministic problems and problems with decomposable structure.

3.6 Population-Based Search

These algorithms maintain a population of candidate solutions and evolve them toward better solutions — inspired by natural processes.

- **Genetic Algorithms:** Evolve solutions using selection, crossover, and mutation operators. Good for optimisation problems.
- **Particle Swarm Optimisation:** Particles (candidate solutions) move through solution space influenced by their personal best and global best.
- **Key advantage:** Explores large search spaces in parallel; avoids local optima better than hill climbing.

4. Adversarial Search — Game Playing

4.1 Introduction to Adversarial Search

In competitive environments, two agents work against each other. These are called adversarial search problems or games. AI must plan ahead, accounting for the opponent's best possible moves.

- Typically modelled as zero-sum games: one player's gain = other's loss.
- Examples: Chess, Tic-Tac-Toe, Checkers, Go.

4.2 Minimax Algorithm

Minimax is the foundational algorithm for two-player zero-sum games. One player is MAX (tries to maximise score) and the other is MIN (tries to minimise score).

How Minimax Works:

1. Build the complete game tree from the current state to terminal states.
2. Assign utility values to terminal states (win = +1, lose = -1, draw = 0).
3. Propagate values upward: MAX nodes take the maximum child value; MIN nodes take the minimum.
4. The root value gives the best achievable outcome; the corresponding move is chosen.

Property	Detail
Completeness	Yes — explores entire game tree
Optimality	Optimal against a perfect opponent
Time Complexity	$O(b^m)$ — b = branching factor, m = game depth
Space Complexity	$O(bm)$
Limitation	Exploring all nodes is exponentially expensive for complex games

4.3 Alpha-Beta Pruning

Alpha-Beta Pruning optimises Minimax by cutting off branches that cannot possibly affect the final decision — dramatically reducing the number of nodes evaluated.

Two key values:

- **Alpha (α):** Best (highest) value that MAX can guarantee so far along the path. Initialised to $-\infty$.
- **Beta (β):** Best (lowest) value that MIN can guarantee so far along the path. Initialised to $+\infty$.

Pruning rule:

- Prune (cut off) a branch when $\alpha \geq \beta$ — MAX will never choose this branch because MIN will not allow it.

Step-by-step:

1. Run Minimax with α and β tracked at each node.
2. At a MAX node: update $\alpha = \max(\alpha, \text{child value})$. If $\alpha \geq \beta$, prune remaining children.
3. At a MIN node: update $\beta = \min(\beta, \text{child value})$. If $\beta \leq \alpha$, prune remaining children.

	Minimax	Alpha-Beta
Time (best case)	$O(b^m)$	$O(b^{m/2})$ — doubles search depth!
Nodes evaluated	All nodes	Roughly half (with optimal ordering)
Result	Same as Minimax	Identical — no quality loss

Alpha-Beta Exam Tip

Alpha-Beta does NOT change the result of Minimax — it just reaches it faster by skipping provably irrelevant branches. In the best case (perfect node ordering) it reduces the search from $O(b^m)$ to $O(b^{m/2})$, effectively doubling the depth AI can search in the same time.

► **Minimax & Alpha-Beta Video:** <https://www.youtube.com/watch?v=HVzr8ZeTR6o>

5. Quick Revision — Unit I at a Glance

5.1 All Search Algorithms Compared

Algorithm	Strategy	Complete?	Optimal?	Time	Space
DFS	Uninformed	No*	No	$O(b^m)$	$O(bm)$
BFS	Uninformed	Yes	Yes*	$O(b^d)$	$O(b^d)$
Best First	Informed	No	No	$O(b^m)$	$O(b^m)$
A*	Informed	Yes	Yes*	Exp.	Exp.
Hill Climb	Local	No	No	—	$O(b)$
AO*	AND-OR	Yes	Yes	Exp.	Exp.
Minimax	Adversarial	Yes	Yes	$O(b^m)$	$O(bm)$
Alpha-Beta	Adversarial	Yes	Yes	$O(b^{m/2})$	$O(bm)$

* Optimal for BFS only when all step costs are equal. * A* optimal with admissible heuristic. * DFS complete with visited list.

5.2 Five Agent Types — One-Liners

- **Simple Reflex:** If condition → then action. No memory.
- **Model-Based Reflex:** Like Simple Reflex but maintains internal world state.
- **Goal-Based:** Uses goals to decide action — can plan ahead.
- **Utility-Based:** Maximises a utility (happiness) function — handles trade-offs.
- **Learning Agent:** Improves over time via learning element + critic + performance element.

5.3 Key Formulae & Facts

- **A* evaluation:** $f(n) = g(n) + h(n)$
- **Admissibility condition:** $h(n) \leq h^*(n)$ [h^* = true cost]
- **Alpha-Beta best case:** $O(b^{(m/2)})$ — doubles effective search depth
- **AI Classification:** Narrow < General < Super
- **Dartmouth Conference year:** 1956 — official birth of AI
- **Deep Blue defeated Kasparov:** 1997

5.4 Video Resources

- ▶ **DFS Explained:** <https://www.youtube.com/watch?v=dWpvK-8uAqM>
- ▶ **BFS Explained:** <https://www.youtube.com/watch?v=SZ2a668G6vA>
- ▶ **AI Agents Overview:** <https://www.youtube.com/watch?v=S7L4-KDTvEE>
- ▶ **Minimax Algorithm:** <https://www.youtube.com/watch?v=HVzr8ZeTR6o>
- ▶ **Alpha-Beta Pruning:** <https://www.youtube.com/watch?v=hHRWRa3k6Ww>
- ▶ **Game Playing (extended):** https://www.youtube.com/watch?v=Q_15qjqX-RE